

Introduce yourself?

Hi good afternoon. Myself Bharati mudi. Aspiring data scientist with 1.5 years of experience in developing end-to-end Solutions using generative AI, ML, CV and NLP. Experienced in building, deploying and optimizing models on cloud platforms to solve real world problems.

For studies, I completed my masters from bangalore university in 2024 and bachelors from kuvempu university in 2022. I am originally from Haveri district rural area in karnataka.

Coming to my work experience, recently I worked as an AI associate at ResourcePro where I as developed AI-driven automation solutions for insurance workflows using ML, GenAI and NLP. Before that I completed 2 interships as a Data Analyst intern at Ai4see PVT LTD and Data science intern at iNeuron intelligence pvt Ltd, which helped me to build a strong foundation in the business industry.

Apart from this I worked some projects. I have built end to end AI solutions in my key projects being a medical chatbot using Llama3, Langchain, pinecone and RAG architecture. I also worked on AI agents workflows using langGraph. A source code analysis tool using genAI, a Book Recommendation system and a fire and smoke detection system using Yolov11 model. Along with this way I gained hands on experience with python SQL langchain, langGraph, vector databases as pinecone weaviate, and chromaDB, langsmith Ragas, DeepEval, FastAPI, docker and deploying applications on AWS and GCP. What I enjoy most building AI systems that solve real business problems. Now I am looking for an opportunity where I can contribute to designing scalable, reliable AI solutions while continuing to learn and grow as a genAI engineer.

Explain about Medical Chatbot

Medical Chatbot, This project i built using a Retrieval-Augmented Generation (RAG) architecture. The goal was to answer medical questions accurately using trusted medical documents containing information about diseases, diagnoses, and treatments.

First, I extracted the medical content, split it into smaller chunks, generated embeddings for each chunk, and stored them in Pinecone, a cloud-based vector database. I chose Pinecone because it efficiently handles large datasets and provides fast semantic similarity search.

When a user asks a question, the query is converted into an embedding, and Pinecone retrieves the Top-K most relevant document chunks. Using LangChain, I passed the retrieved context along with the user's question to the LLM, which generated an answer based only on the retrieved information. This RAG approach helped reduce hallucinations and improved response accuracy.

For the application, I used Flask for the user interface, followed a modular Python code structure, and deployed it on AWS using a CI/CD pipeline. I also used LangSmith for tracing and debugging, and Ragas to evaluate retrieval quality and answer faithfulness

Explain Sorce code analysis system

Source Code Analysis System built using Generative AI. The objective was to automatically analyze Python source code and generate a structured understanding of the codebase, making it easier for developers to understand large projects and improve documentation.

The system first parses Python files using the Abstract Syntax Tree (AST) module to extract structural information such as classes, functions, imports, variables, and their relationships. I also used regular expressions for additional pattern matching where AST alone was not sufficient.

The extracted information is then organized into a structured format and passed to an LLM, which generates summaries, explains the code logic, identifies dependencies, and highlights potential issues such as unused functions or coding patterns.

I developed the backend in Python, used Pandas for processing the extracted data, and built the user interface with Streamlit. The system helped improve code understanding, reduced manual code review effort, and could be extended for documentation generation or onboarding new developers

Explain your Book Recommendation System.

Book Recommendation System this project recommends books based on a user's reading preferences using collaborative filtering. The goal was to provide personalized recommendations by identifying users with similar interests rather than relying only on book content.

I used a dataset containing user IDs, book IDs, and ratings. After preprocessing the data, I created a user-item interaction matrix and applied the K-Nearest Neighbors (KNN) algorithm to find users or books with similar rating patterns. Based on these similarities, the system recommends books that similar users have highly rated.

I developed the solution using Python, Pandas, and Scikit-learn, and deployed the model on AWS SageMaker. I also automated deployment using GitHub Actions. The same recommendation approach can be extended to other domains such as movies, e-commerce, and music recommendations.

Explain Fire and smoke detection project

Real-time Fire and Smoke Detection System developed using the YOLOv11 object detection model. The goal was to detect fire and smoke at an early stage from images or live video streams to improve safety in environments such as factories, offices, and warehouses.

I first collected and annotated a custom dataset using Roboflow, where each image was labeled with fire and smoke bounding boxes. After preprocessing and augmentation, I trained the YOLOv11 model on the dataset.

During inference, the model processes each image or video frame, detects the presence of fire or smoke, and draws bounding boxes with confidence scores. I used OpenCV for real-time video processing and displaying the detection results.

The system achieved accurate real-time detection and can be integrated with alert systems like alarms, SMS, or email notifications for early fire detection

What is Agent and how do you implement in your project?

An AI agent is an LLM-powered system that can reason, decide which tools to use, execute actions, and generate responses autonomously. In my Medical Chatbot, I implemented a LangGraph-based agent that first decided whether document retrieval was needed, used Pinecone to fetch relevant medical information, and then passed that context to the LLM to generate an accurate, grounded response.

Ex: In my Medical Chatbot project, I implemented an agent using LangGraph to make the workflow more intelligent and modular. Instead of directly sending every user query to the LLM, the agent first analyzed the user's request and decided the next action.

For medical questions, the agent invoked the retriever tool to fetch relevant information from the Pinecone vector database. After retrieving the Top-K relevant medical documents, it passed both the retrieved context and the user's question to the LLM to generate a grounded response. If no relevant documents were found, the agent returned an appropriate message instead of generating a potentially incorrect answer.

Using LangGraph allowed me to build a state-based workflow with clear control over retrieval, response generation, and error handling, making the system more reliable and easier to extend